

Frontend Migration Guide (Examples + Rationale)

Scenario: Your legacy Android frontend is built with **Java + XML** (Activity/Fragment + RecyclerView + DataBinding/ViewBinding). You plan to migrate to **Android: Jetpack Compose** and **iOS: SwiftUI**, while business logic and data access move into the **KMM shared module** (Domain + Data). This guide covers migration principles, phased plan, component mapping, key coding patterns, interop, performance & accessibility, testing & acceptance. **Each example includes a short “Rationale: Why this way”.**

0. Success Criteria & Milestones

- **Feature parity:** Anything the old Android UI could do must keep working—no regressions.
 - **Cross-platform parity:** For critical user journeys (sign-in, list, details, submit), Android and iOS behave consistently.
 - **Single Source of Truth:** UI renders from UiState only; business rules live in shared use cases.
 - **Rollbackable:** Major screens are gated behind feature flags to allow quick fallback to legacy.
 - **Test coverage:** Compose UI tests + JVM unit tests cover core use cases.
-

1. Frontend Architecture & Project Layout

```
androidApp/  
  ui/  
    feature/login/  
      LoginScreen.kt  
      LoginViewModel.kt  
      LoginNav.kt  
    feature/home/  
    feature/users/  
  designsystem/ # Tokens/Theme and reusable components  
  navigation/   # App-level NavHost + multi back-stack support
```

```
iosApp/  
  Features/  
    LoginView.swift  
    UsersListView.swift  
  ViewModels/  
    LoginVM.swift  
  Theme/  
  Navigation/
```

Rationale: Organize by **feature** so each function can migrate/rollback independently. Keep a dedicated **design system** to avoid scattered styles.

2. State Management Baseline (MVVM + StateFlow)

UiState model

```
// Android/KMM consensus: loading + data + error
```

```
data class UsersUiState(  
    val loading: Boolean = false,  
    val items: List<User> = emptyList(),  
    val error: String? = null  
)
```

ViewModel (Android)

```
class UsersViewModel(private val getUsers: GetUsers): ViewModel() {  
    private val _ui = MutableStateFlow(UsersUiState())  
    val ui: StateFlow<UsersUiState> = _ui  
  
    fun refresh() = viewModelScope.launch {  
        _ui.value = _ui.value.copy(loading = true, error = null)  
        _ui.value = getUsers().fold(  
            onSuccess = { UsersUiState(items = it) },  
            onFailure = { UsersUiState(error = it.message) }  
        )  
    }  
}
```

Compose subscription

```
@Composable
fun UsersScreen(vm: UsersViewModel = koinViewModel()) {
    val state by vm.ui.collectAsStateWithLifecycle()
    when {
        state.loading -> CircularProgressIndicator()
        state.error != null -> ErrorView(state.error!!) { vm.refresh() }
        else -> UsersList(state.items)
    }
}
```

Rationale: `UiState` is the **SSoT**. UI does not run network/validation directly. SwiftUI mirrors the same pattern using `ObservableObject` + `@Published`.

3. XML → Compose Mapping Cheat Sheet

XML / Legacy Component	Compose Counterpart	Example Section
RecyclerView + Adapter	LazyColumn / LazyRow	§ 4
ConstraintLayout	ConstraintLayout-Compose or Box/Row/Column	§ 5
TextInputLayout + EditText	OutlinedTextField / TextField	§ 6
Toolbar/AppBar	TopAppBar / CenterAlignedTopAppBar	§ 7
BottomNavigationView	NavigationBar + Navigation Compose	§ 8
ProgressBar	Circular/LinearProgressIndicator	—
Dialog / BottomSheetDialog	AlertDialog / ModalBottomSheet	§ 9
ViewPager2 + TabLayout	TabRow + HorizontalPager (Accompanist)	—
SwipeRefreshLayout	PullToRefreshBox or Accompanist	—

4. Lists: RecyclerView → LazyColumn (with Paging)

Legacy (simplified)

```
class UserAdapter: ListAdapter<User, VH>(DIFF) { ... }
```

Compose + Paging3

```
@Composable
fun UsersList(paging: LazyPagingItems<User>) {
    LazyColumn {
        items(paging.itemCount, key = paging.itemKey { it.id }) { idx ->
            paging[idx]?.let { u -> ListItem(headlineText =
                { Text(u.name) }) }
        }
        paging.apply {
            when {
                loadState.append is LoadState.Loading -> item
            } { LinearProgressIndicator() }
        }
    }
}
```

Rationale: Eliminates Adapter/ViewHolder boilerplate; **stable keys** replace DiffUtil; Paging3's Compose adapter makes paging first-class (consistent loading/error/retry).

5. Layouts: Two Ways to Replace ConstraintLayout

A. Keep ConstraintLayout-Compose (for complex constraints)

```
ConstraintLayout(Modifier.fillMaxSize()) {
    val (avatar, name, btn) = createRefs()
    Image(avatarPainter, null,
        Modifier.constrainAs(avatar) { start.linkTo(parent.start);
            top.linkTo(parent.top) })
}
```

```

    Text(user.name,
Modifier.constrainAs(name) { start.linkTo/avatar.end, 8.dp);
top.linkTo/avatar.top) })
    Button(onClick={},
Modifier.constrainAs(btn) { end.linkTo(parent.end);
baseline.linkTo(name.baseline) }) { Text("Follow") }
}

```

B. Refactor using Box/Row/Column (for simple flows)

```

Row(Modifier.fillMaxWidth(), verticalAlignment =
Alignment.CenterVertically) {
    Image/avatarPainter, null)
    Column(Modifier.weight(1f).padding(start = 8.dp))
{ Text(user.name); Text(user.title) }
    Button(onClick = {}) { Text("Follow") }
}

```

Rationale: Prefer simplicity. Linear layouts with Row/Column are more readable and often faster; use ConstraintLayout-Compose only when constraints are truly needed.

6. Forms: TextInputLayout →

OutlinedTextField (Validation in Shared)

Legacy (XML + DataBinding expression)

```
<Button ... android:enabled="@{vm.emailValid && vm.pwdLen >= 8}"/>
```

Compose + Use Case

```

class ValidateLogin { operator fun invoke(e:String, p:String)=
e.contains('@') && p.length>=8 }

```

@Composable

```

fun LoginForm(vm: LoginViewModel = koinViewModel()) {
    var email by rememberSaveable { mutableStateOf("") }
    var pwd by rememberSaveable { mutableStateOf("") }
    val canSubmit = remember(email, pwd) { ValidateLogin() (email,
pwd) }
    OutlinedTextField(email, { email = it }, label={Text("Email")})
}

```

```

        OutlinedTextField(pwd, { pwd = it },
visualTransformation=PasswordVisualTransformation(),
label={Text("Password")})
        Button(onClick = { vm.login(email, pwd) }, enabled = canSubmit)
    { Text("Login") }
}

```

Rationale: Move validation into **shared** so Android & iOS reuse the same rules. Compose no longer embeds complex XML expressions.

7. Top Bar & Menus: Toolbar → TopAppBar

```

@Composable
fun ScreenTopBar(onBack: ()->Unit, onRefresh: ()->Unit) {
    TopAppBar(
        title = { Text("Users") },
        navigationIcon =
    { IconButton(onClick=onBack) { Icon(Icons.AutoMirrored.Filled.ArrowBack, null) } },
        actions =
    { IconButton(onClick=onRefresh) { Icon(Icons.Default.Refresh, null) } }
    )
}

```

Rationale: Events are **lambdas**, removing `onOptionsItemSelected` boilerplate. Easy to test and mirror on iOS `NavigationBar`.

8. Bottom Navigation: BottomNavigationView → NavigationBar (Multi-stack)

```

@Composable
fun AppNav() {
    val nav = rememberNavController()
    val items = listOf("home", "users", "profile")
    Scaffold(bottomBar = {
        NavigationBar {
            items.forEach { route ->
                NavigationBarItem(

```

```

        selected = currentRoute(nav) == route,
        onClick = {
            nav.navigate(route) {
                launchSingleTop = true
                restoreState = true
                popUpTo(nav.graph.startDestinationId) { saveState =
true }
            }
        },
        icon = { Icon(Icons.Default.Home, null) },
        label = { Text(route) }
    )
}
}) { pad ->
    NavHost(nav, startDestination = "home", Modifier.padding(pad)) {
        composable("home") { HomeScreen() }
        navigation(startDestination = "users/list", route = "users") {
            composable("users/list") { UsersScreen() }
            composable("users/detail/{id}") { backStackEntry -> /* ...
*/ }
        }
        composable("profile") { ProfileScreen() }
    }
}
}

```

Rationale: Each tab **retains its own back stack**, matching legacy UX; supports deep links and state restoration.

9. Dialogs: DialogFragment → AlertDialog (State-driven)

```

@Composable
fun
ConfirmDelete(show:Boolean, onConfirm: ()->Unit, onDismiss: ()->Unit) {
    if (show) AlertDialog(
        onDismissRequest=onDismiss,
        title={ Text("Delete") }, text={ Text("Are you sure?") },
        confirmButton={ TextButton(onClick=onConfirm) { Text("OK") } },
        dismissButton={ TextButton(onClick=onDismiss) { Text("Cancel") } }
    )
}

```

```
)  
}
```

Rationale: Boolean-controlled visibility avoids Fragment transaction timing pitfalls; mirrors SwiftUI.

10. Interop (Essential for Gradual Rollout)

Compose inside a Fragment

```
class ProfileFragment: Fragment(R.layout.fragment_profile) {  
    override fun onCreateView(v:View, s:Bundle?) {  
        v.findViewById<ComposeView>(R.id.compose).setContent  
        { ProfileScreen() }  
    }  
}
```

Legacy View inside Compose (e.g., Map/WebView/Player)

```
@Composable  
fun LegacyMap(modifier: Modifier = Modifier) {  
    AndroidView(  
        factory = { ctx -> MapView(ctx).apply{ onCreate(null);  
onResume() } },  
        update = { /* update camera/markers */ },  
        modifier = modifier,  
        onRelease = { it.onPause(); it.onDestroy() }  
    )  
}
```

Rationale: Enables **incremental rollout** with safe rollback. Use onRelease/DisposableEffect to free resources and prevent leaks.

11. IME & Focus

```
val focus = LocalFocusManager.current  
OutlinedTextField(  
    value = text, onValueChange = { text = it },  
    keyboardOptions = KeyboardOptions(imeAction = ImeAction.Done),
```



```
keyboardActions = KeyboardActions(onDone = { focus.clearFocus() })
)
```

Rationale: IME behavior is declarative and testable; no scattered listeners.

12. Theming & Design System (Material 3)

Theme baseline

```
@Composable
fun AppTheme(content:@Composable ()->Unit){
    val dark = isSystemInDarkTheme()
    val scheme = if (dark) darkColorScheme() else lightColorScheme()
    MaterialTheme(colorScheme=scheme, typography=AppTypography,
    shapes=AppShapes, content=content)
}
```

Ripple/Selector migration

```
Button(
    onClick = {},
    modifier = Modifier.indication(remember
{ MutableInteractionSource() }, rememberRipple())
){ Text("OK") }
```

Rationale: Centralize brand tokens and adaptation in Theme. Replace XML selectors with interaction source + ripple to reduce style fragmentation.

13. Resources & Localization

```
Text(stringResource(R.string.welcome))
// If using Compose Multiplatform resources:
org.jetbrains.compose.resources
```

Rationale: Keep existing strings.xml and consume from Compose. Only consider shared resources if you truly need cross-platform assets; generally keep UI resources native.

14. Accessibility (A11y)

```
Image(painter, contentDescription = stringResource(R.string.avatar))
Box(Modifier.semantics { stateDescription = if (selected) "Selected"
else "Not selected" }) { /* ... */ }
```

Rationale: Declarative semantics are more controllable than scattered `setContentDescription`; SwiftUI has `accessibilityLabel` equivalents.

15. Animation & Gestures

```
val expanded by remember { mutableStateOf(false) }
AnimatedVisibility(visible = expanded) { Text("Details") }
val offsetY by animateDpAsState(if (expanded) 0.dp else 200.dp)
Box(Modifier.offset(y = offsetY)) { /* ... */ }
```

Rationale: **Data-driven animation** prevents manual tweening/state drift; iOS can use the same state rules.

16. Performance & Recomposition Control

- Provide stable keys; cache expensive work with `remember/derivedStateOf`.
- Avoid creating heavy objects inside items `{}`.
- Proper image sizing/caching (Coil-Compose).
- Use `snapshotFlow { listState.firstVisibleItemIndex }` for scroll observation instead of causing frequent recompositions.

Rationale: Make **reuse** and **caching** explicit to minimize unnecessary recompositions.

17. Permissions & Activity Result API

```
val launcher = rememberLauncherForActivityResult(  
    ActivityResultContracts.RequestPermission()  
) { granted -> /* ... */ }  
Button(onClick = { launcher.launch(Manifest.permission.CAMERA) })  
{ Text("Grant Camera") }
```

Rationale: Standardize on **Activity Result API**; retire legacy callbacks.

18. Testing & Debugging

Compose UI test

```
@Test fun login_button_enabled_when_valid() {  
    composeTestRule.setContent { LoginForm() }  
    onNodeWithText("Email").performTextInput("a@b.com")  
    onNodeWithText("Password").performTextInput("12345678")  
    onNodeWithText("Login").assertIsEnabled()  
}
```

Debug recomposition: Layout Inspector, *Show Layout Bounds*; avoid I/O inside @Composable.

Rationale: Semantic selectors are more stable than withId.
Declarative UI is friendlier to automation.

19. SwiftUI Co-migration Notes

```
final class LoginVM: ObservableObject {  
    @Published var state = LoginState()  
    func login(email: String, pwd: String) { /* call shared use case  
and update state */ }  
}
```

```
struct LoginView: View {
```

```

@StateObject var vm = LoginVM()
var body: some View {
    VStack {
        TextField("Email", text: $vm.state.email)
        SecureField("Password", text: $vm.state.pwd)
        Button("Login") { vm.login(email: vm.state.email, pwd:
vm.state.pwd) }
        .disabled(!vm.state.canSubmit)
    }
}
}

```

Rationale: SwiftUI' s ObservableObject + @Published aligns naturally with Compose patterns. Validation/use cases come from shared for cross-platform consistency.

20. Common Pitfalls & Fixes

1. **TextField cursor/ghosting:** Missing remember/rememberSaveable; ensure the value is a stable state.
 2. **Dialog won' t dismiss:** Missing onDismissRequest or state not toggled.
 3. **AndroidView leaks:** Not releasing resources in onRelease/DisposableEffect.
 4. **Res.* not generated:** Wrong resource paths/names or plugin not enabled.
 5. **Excess recomposition:** No stable key or new objects created inside Composables.
-

Closing Note

This guide emphasizes **state-centric UI**, **incremental rollout**, and **shared business rules**. Get the critical path working with a safe rollback, then replace the rest—including complex widgets. Push all business/validation into shared to ensure **consistency**, **testability**, and **maintainability** across Android and iOS.

References

JetBrains. *Kotlin Multiplatform / Kotlin Documentation* — KMP overview and goals.

Google Android Developers. *Kotlin Multiplatform Overview* — Google's endorsement and getting started.

Google Android Developers. *Guide to app architecture* — SSOT/UDF principles.

Google Android Developers. *Jetpack Compose performance* — recomposition/perf guidance.

Google Android Developers. *Interoperability APIs (Views ↔ Compose)* — key APIs for gradual migration.

Google Android Developers. *Navigation with Compose* — official navigation docs.

Apple Developer Documentation. *NavigationStack* — SwiftUI nav stack/deeplink behavior.

JetBrains Docs. *Create a multiplatform app using Ktor and SQLDelight* — shared data layer pattern with native UI.

Ktor Documentation. *Creating a cross-platform mobile application* — Ktor client multiplatform.

SQLDelight Docs. *Getting Started - Android SQLite (v2.0.2)* — multiplatform driver details.

Koin Official. *Koin — Multiplatform* — DI across KMM.

Russell Wolf. *Multiplatform Settings — README* — cross-platform KV store API.

Google Android Developers. *Test your Compose layout* — UI test & semantics.

Guspiel, M. (2024). *Mobile App Development Using Kotlin Multiplatform*. Theseus Thesis.

KTH DiVA (2025). *Performance Evaluation of Kotlin Multiplatform on iOS: Quantitative Study.*

Umeå University DiVA (2025). *Kotlin Multiplatform vs Native Development: A Performance Analysis for Mobile Apps.*

MDPI (2025). *State-of-the-Art Cross-Platform Mobile Application Development: Trends & Adoption.*